UNIT 4 · CO4

Memory Organizations — Student Notes

Computer Organization and Architecture (CSE23CT201) · Sri Ramachandra Faculty of Engineering and Technology
A standalone reading handout — bullet points, figures as scanned textbook plates, a worked example and a question box under each topic. Consolidated single pass over the syllabus: each topic is taught once, primarily from Hamacher, Vranesic & Zaky (Chapter 8). Mano (Chapter 12) is used only where it is the sole or the completing source — Associative Memory, the associative-memory page table, and Memory Management Hardware.

A note on sources. Topics that both textbooks cover are given once, in Hamacher's treatment, to avoid teaching the same idea twice. Where a chapter still cites Mano §12-x, it is because Hamacher has no equivalent (Associative Memory, Memory Management Hardware) or because Mano completes the picture (the associative-memory page table in Virtual Memory). All seven syllabus topics are covered.

Contents

1. Memory Hierarchy
2. Main Memory — RAM and ROM Chips
3. Auxiliary Memory
4. Associative Memory
5. Cache Memory
6. Virtual Memory
7. Memory Management Hardware

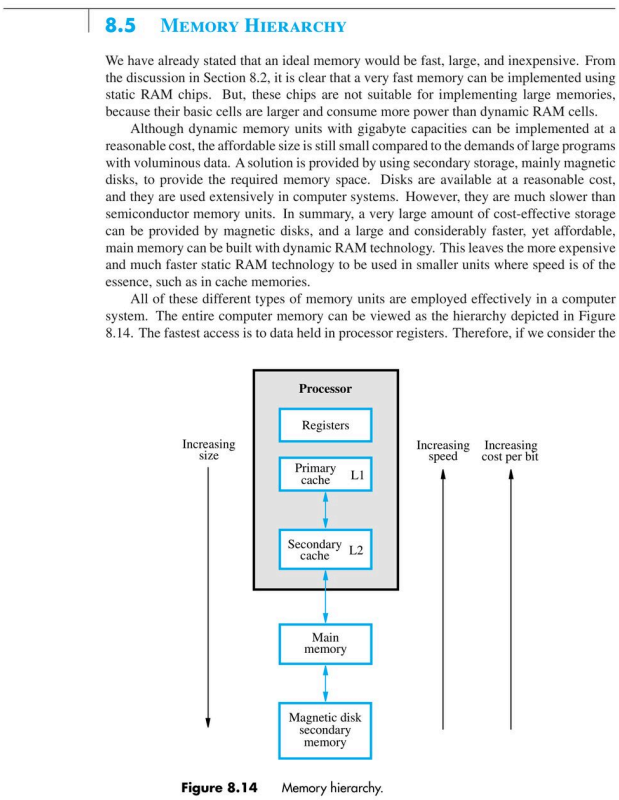
1. Memory Hierarchy

Hamacher §8.5

1.1 Why a hierarchy, and not one big fast memory?

- An ideal memory would be **fast**, **large**, and **inexpensive** — all at the same time. In practice no single technology gives you all three: fast memory (static RAM) cannot be made large and cheap; large, cheap memory (magnetic disk) is slow.
- The way around this is a **memory hierarchy**: several levels of storage of increasing size, increasing access time, and decreasing cost per bit, with each level acting as a partial working store ("cache") for the level below it.
- Levels, fastest/smallest to slowest/largest: **processor registers → primary (L1) cache → secondary (L2) cache → main memory (RAM) → magnetic disk secondary memory**. Mano's version of the same idea (Fig 12-1) draws it from the system side: **magnetic tapes** and **magnetic disks** (auxiliary memory) feed an **I/O processor**, which moves data into **main memory**; the **CPU** talks directly only to main memory and **cache memory**.

Key idea: going down the hierarchy — size **increases**, speed **decreases**, cost per bit **decreases**. The whole scheme only works because of **locality of reference**: programs access a small, localized set of addresses repeatedly within any short interval of time (temporal locality) and tend to reference addresses near ones they just used (spatial locality).



8.5 MEMORY HIERARCHY

We have already stated that an ideal memory would be fast, large, and inexpensive. From the discussion in Section 8.2, it is clear that a very fast memory can be implemented using static RAM chips. But, these chips are not suitable for implementing large memories, because their basic cells are larger and consume more power than dynamic RAM cells.

Although dynamic memory units with gigabyte capacities can be implemented at a reasonable cost, the affordable size is still small compared to the demands of large programs with voluminous data. A solution is provided by using secondary storage, mainly magnetic disks, to provide the required memory space. Disks are available at a reasonable cost, and they are used extensively in computer systems. However, they are much slower than semiconductor memory units. In summary, a very large amount of cost-effective storage can be provided by magnetic disks, and a large and considerably faster, yet affordable, main memory can be built with dynamic RAM technology. This leaves the more expensive and much faster static RAM technology to be used in smaller units where speed is of the essence, such as in cache memories.

All of these different types of memory units are employed effectively in a computer system. The entire computer memory can be viewed as the hierarchy depicted in Figure 8.14. The fastest access is to data held in processor registers. Therefore, if we consider the

Figure 8.14 Memory hierarchy.

Figure 8.14 — Memory hierarchy (Hamacher, Vranesic & Zaky, p.288). Faster levels sit closer to the processor and are smaller; data migrates between levels as needed.

1.2 Access-time and capacity ratios

- The overall goal of a memory hierarchy is to obtain the **highest-possible average access speed** while minimizing the total cost of the entire memory system.
- Typical ratios quoted by Hamacher: a cache memory may have an access time of about **100 ns**, while main memory access time may be about **700 ns** — a **5–10×** speed gap. Auxiliary memory average access time is usually **1000 times** that of main memory.
- Mano notes typical **block sizes**: in auxiliary memory, blocks of data are typically **256 to 2048 words**; cache block size is typically **1 to 16 words**. Transfers between auxiliary and main memory are usually of medium-sized blocks of data; between cache and main memory, of much smaller blocks — but far more often.
- **Multiprogramming**: many operating systems allow several independent programs to reside in different parts of the memory hierarchy concurrently and share the CPU. This is why, at any instant, only a fraction of all resident programs and data can be in main memory — the rest wait in auxiliary memory until the OS brings them in.

WORKED EXAMPLE

Q. A system has cache access time 10 ns, main memory access time 100 ns, and a cache hit ratio of 0.95. Find the average memory access time.

A. Average access time = (hit ratio × cache time) + (miss ratio × (cache time + memory time))
= (0.95 × 10) + (0.05 × (10 + 100)) = 9.5 + 5.5 = **15 ns**.
Without any cache, every access would cost 100 ns; with the cache in place the effective access time drops to 15 ns — illustrating why even a fast, small top level of the hierarchy has such a large effect on overall performance.

CHECK YOUR UNDERSTANDING

1. Why can't a computer be built entirely out of the fastest available memory technology? **[2M]**
2. Sketch the memory hierarchy from registers to secondary storage and state how access time, size, and cost per bit change at each level. **[6M]**
3. Define locality of reference and explain why it justifies a memory hierarchy. **[4M]**
4. What is multiprogramming, and how does it relate to the fact that main memory is much smaller than auxiliary memory? **[4M]**
5. Differentiate auxiliary memory and cache memory in terms of purpose, typical block size, and where each sits relative to main memory. **[6M]**

---

2. Main Memory — RAM and ROM Chips

Hamacher §8.1–8.3

2.1 Connecting memory to the processor

- The maximum size of memory a computer can use is fixed by its **addressing scheme**: a processor generating k-bit addresses can access up to 2^k locations (its **address space**).
- The processor–memory connection has three groups of lines: **address lines** (select the location), **data lines** (carry the word being read/written), and **control lines** (carry Read/Write and timing signals, e.g. R/$\bar{W}$).

2.2 Random-access memory (RAM): static vs dynamic

- **Random-access memory (RAM)** chips are available in two operating modes:
- **Static RAM (SRAM)** stores each bit in a **latch** made of cross-coupled transistors (a flip-flop). It is fast and needs **no refreshing** — the stored value stays valid as long as power is applied — but each cell needs several transistors, so SRAM is used only where speed is essential, such as **cache memory**.
- **Dynamic RAM (DRAM)** stores each bit as **charge on a capacitor**, accessed through a single transistor. Because the charge leaks away, DRAM must be **periodically refreshed** (every few milliseconds) by reading and rewriting each cell. DRAM cells are smaller and cheaper, so DRAM is used to build the bulk of **main memory**.
- Address pins on a DRAM chip are commonly **multiplexed** into a **row address** and a **column address** (RAS/CAS — Row/Column Address Strobe) to reduce the pin count needed on the package.

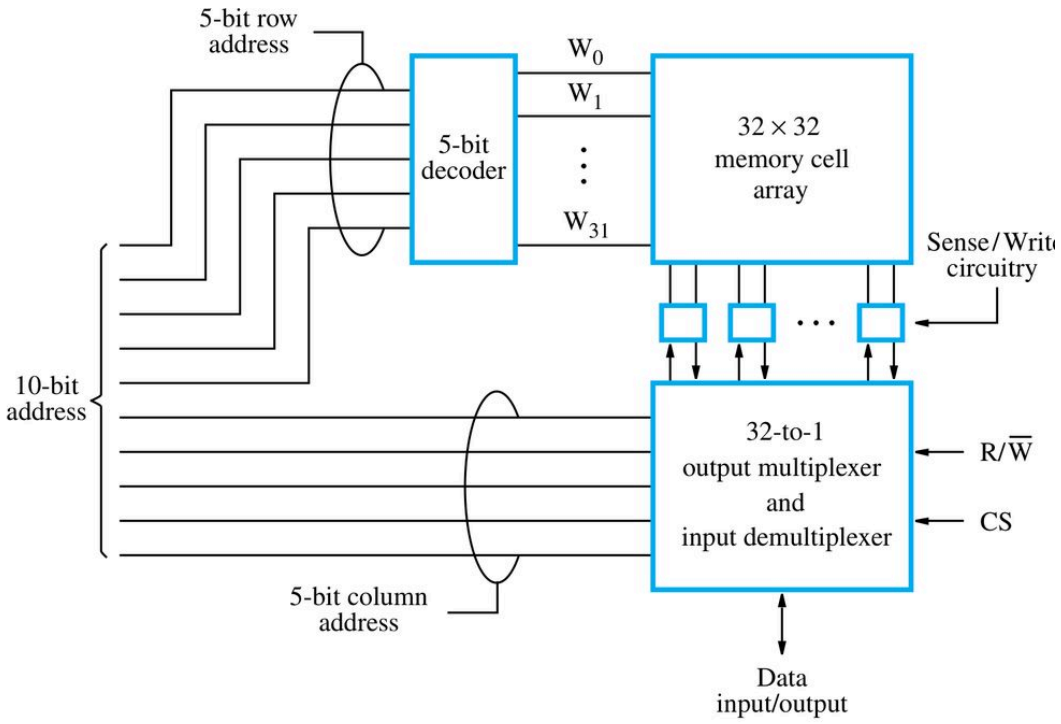


Figure 8.3 Organization of a 1K × 1 memory chip.

Figure 8.3 — Organization of a 1K × 1 memory chip (Hamacher, p.272). A 10-bit address is split into a 5-bit row and 5-bit column; the decoder selects one of 32 word lines, and the 32-to-1 multiplexer/demultiplexer selects the single bit within that row.

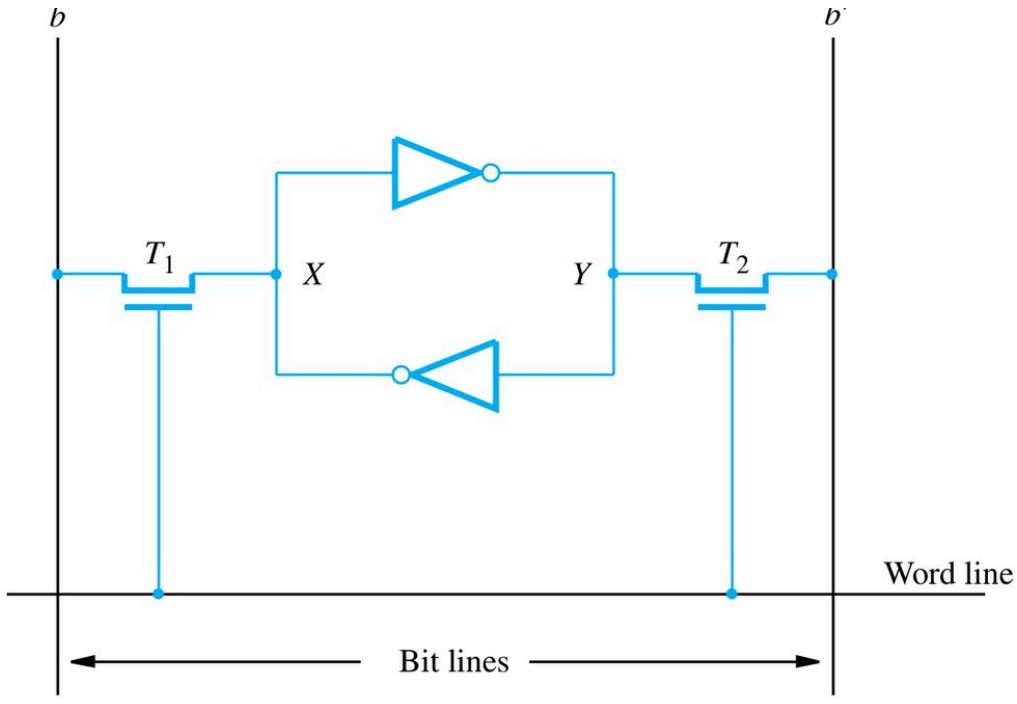


Figure 8.4 A static RAM cell.

Figure 8.4 — A static RAM cell (Hamacher, p.272). Two cross-coupled inverters hold the bit; transistors T1/T2 connect the cell to the bit lines when its word line is asserted.

2.3 Read-only memory (ROM) and its variants

- **Read-only memory (ROM)** holds data permanently: it is written once (usually during manufacture) and only read afterwards. Unlike RAM, ROM is **non-volatile** — it retains its contents when power is removed — which is why it is used to hold a computer's startup/**bootstrap loader** program.
- Variants, in increasing order of flexibility: **PROM** (programmed once by the user with special equipment), **EPROM** (erasable under ultraviolet light, then reprogrammable), and **EEPROM/Flash** (erasable and reprogrammable electrically, in-circuit) — Flash is the basis of modern SSDs and USB drives.
- A ROM chip needs no read/write control input (it can only be read), so for the same word count it needs slightly fewer control pins than an equivalent RAM chip — Mano's example uses a **512 × 8 ROM** alongside 128×8 RAM chips, needing only CS1, CS2, a 9-bit address, and the data bus.

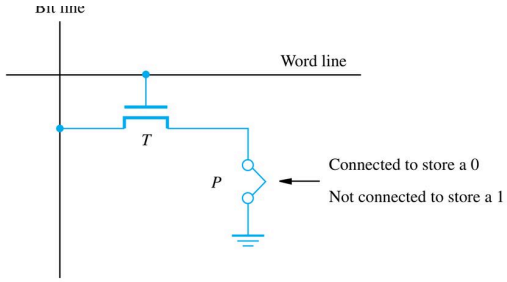


Figure 8.11 A ROM cell.

ROM

y is called a read-only memory, or ROM, when information can be written into
ce at the time of manufacture. Figure 8.11 shows a possible configuration for a
l. A logic value 0 is stored in the cell if the transistor is connected to ground at
otherwise, a 1 is stored. The bit line is connected through a resistor to the power
o read the state of the cell, the word line is activated to close the transistor switch.
lt, the voltage on the bit line drops to near zero if there is a connection between
stor and ground. If there is no connection to ground, the bit line remains at the
ige level, indicating a 1. A sense circuit at the end of the bit line generates the
tput value. The state of the connection to ground in each cell is determined when
s manufactured, using a mask with a pattern that represents the information to be

PROM

M designs allow the data to be loaded by the user, thus providing a *programmable*
OM). Programmability is achieved by inserting a fuse at point *P* in Figure 8.11.
is programmed, the memory contains all 0s. The user can insert 1s at the required

Figure 8.11 — A ROM cell (Hamacher, p.283). A transistor at a bit/word-line crossing is permanently present (storing a 0) or absent (storing a 1); the pattern is fixed at manufacture.

2.4 Building a larger memory module from chips, and the memory address map

- Real memories are built by combining many chips: chips are placed **side by side** to widen the data word, and in **banks** selected by extra address bits to increase the number of locations.
- Mano's numerical example: a computer needs **512 bytes of RAM** and **512 bytes of ROM**, built from 128×8 RAM chips and 512×8 ROM chips. Since $512 = 2^9$, a 9-bit address is required; the two highest-order address lines (8 and 9) are decoded to select among 4 RAM chips and the 1 ROM chip — this table of chip-to-address assignment is called a **memory address map**.

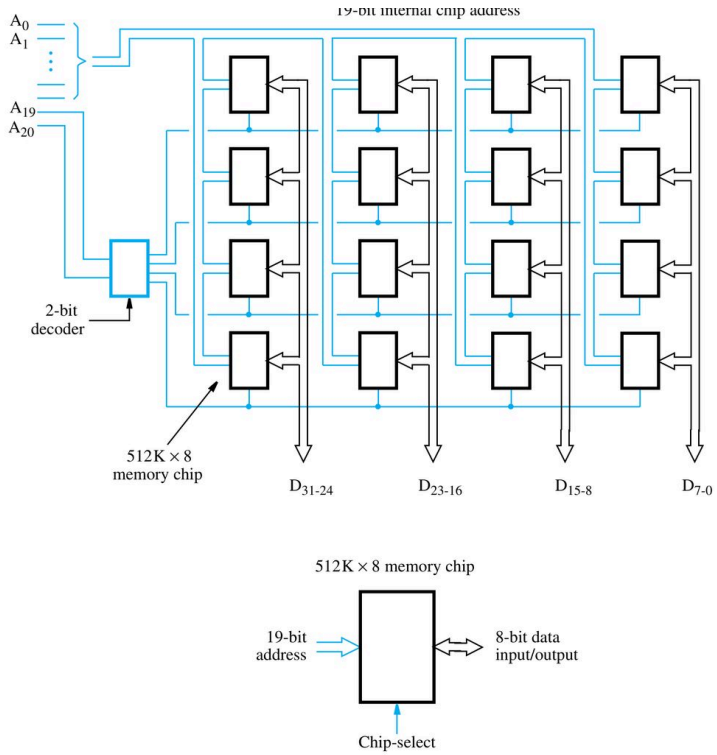


Figure 8.10 Organization of a 2M × 32 memory module using 512K × 8 static memory chips.

Figure 8.10 — Organization of a 2M × 32 memory module using 512K × 8 static memory chips (Hamacher, p.280). Four chips side-by-side supply the 32-bit word; a 2-bit decoder selects among banks to reach the full 2M-word address range.

WORKED EXAMPLE

Q. (Mano, adapted) A computer employs RAM chips of 256 × 8 and ROM chips of 4K × 8. The system needs 2K bytes of RAM and 4K bytes of ROM. How many RAM and ROM chips are needed, and how many address lines are required?

A. RAM: 2048 / 256 = **8 RAM chips**. ROM: one 4K×8 chip already supplies exactly 4K bytes, so **1 ROM chip** is needed. Total address space to cover = 2K + 4K = 6K, which needs **13 address lines** (2^{13} = 8192 ≥ 6144); the low-order bits address a location within a chip (8 bits for RAM, 12 for ROM) and the remaining high-order bits, decoded, select which chip is active.

CHECK YOUR UNDERSTANDING

1. Differentiate SRAM and DRAM in terms of how a bit is stored, and explain why each is used where it is. **[6M]**
2. Why must DRAM be refreshed but not SRAM? **[2M]**
3. List the ROM variants (PROM, EPROM, EEPROM/Flash) and one distinguishing feature of each. **[4M]**
4. A computer uses RAM chips of 1024 × 1 capacity. How many chips are needed, and how should their address lines be connected, to provide a memory capacity of 1024 bytes? **[6M]**
5. Explain the purpose of chip-select inputs and the high-impedance state on a bidirectional data bus. **[4M]**

---

3. Auxiliary Memory

Hamacher §8.9–8.11 (disks/optical/tape)

3.1 What auxiliary memory is for

- **Auxiliary (secondary) memory** provides large-capacity, low-cost, **non-volatile** storage outside the main-memory address space. The most common devices are **magnetic disks** and **magnetic tapes**; less common ones include magnetic drums, magnetic bubble memory, and optical disks.

- Access to a storage location involves an **access time**: for disks/tapes with moving parts, this is a **seek time** (positioning the read/write head over the right track) plus a **transfer time** (moving the data). A **record** is a specified number of characters or words; **transfer rate** is the number of characters/words the device can transfer per second once transfer begins.

3.2 Magnetic disks

- A magnetic disk is a circular plate coated with magnetizable material. Bits are recorded as magnetic spots along concentric circles called **tracks**, which are divided into **sectors** — the smallest unit of information that can be transferred to/from the disk.
- The disk rotates at uniform speed; a **read/write head**, held on an **access mechanism**, moves radially to reach the desired track, then waits for the target sector to rotate underneath it.
- Total access time = **seek time** (move head to the track) + **rotational latency** (wait for the sector) + **transfer time** (read the data as it passes under the head).
- **Hard disks** have platters permanently fixed to the drive; a **floppy disk** uses small, removable disks (5.25″ or 3.5″) coated with magnetic material and was once common for distributing software.

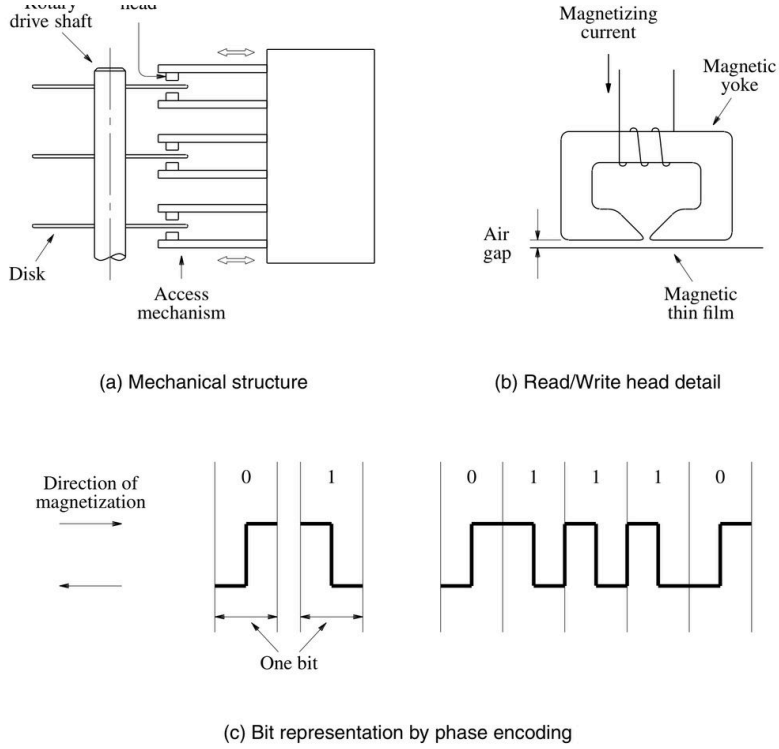


Figure 8.27 Magnetic disk principles.

Figure 8.27 — Magnetic disk principles (Hamacher, p.312): mechanical structure with rotary drive shaft, read/write head and access mechanism (a); read/write head detail (b); bit representation by phase encoding on the track (c).

3.3 Optical disks and magnetic tape

- **Optical disks** (CD, DVD, Blu-ray) encode data as microscopic **pits and lands** along a single, very long **spiral track**, read by reflecting a laser off the surface. They provide high storage density at low cost per bit, though generally slower random access than magnetic disks.
- **Magnetic tape** is a **sequential-access** medium: a strip of plastic coated with magnetic recording material, read/written a block at a time as it winds past a stationary head. Tape units can be started, stopped, and reversed, but individual characters cannot be started or stopped fast enough — hence data is written in **blocks** separated by gaps of unrecorded tape.
- Because it must be searched sequentially from wherever the tape currently sits, tape has by far the **slowest average access time** of any common storage device, but the **lowest cost per bit** — which is why it remains popular for backup/archival storage.

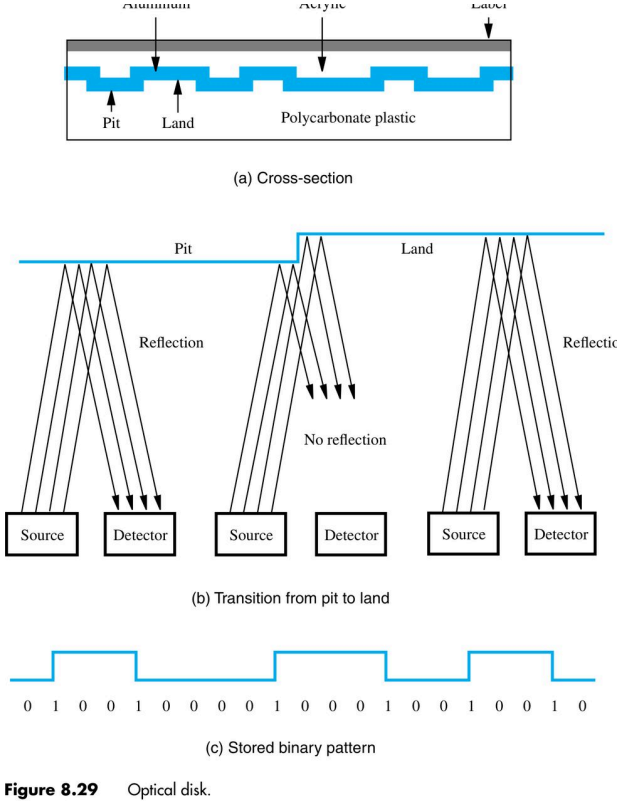


Figure 8.29 Optical disk.

Figure 8.29 — Optical disk (Hamacher, p.318): data encoded as pits along a single spiral track, read by a laser/lens/photodetector assembly.

WORKED EXAMPLE

Q. (Mano, adapted) A magnetic disk system has: T_s = average seek time, R = rotation speed (revolutions per second), N_t = bits per track, N_s = bits per sector. Give an expression for the average time T_a to read one sector.

A. $T_a = T_s$ + (average rotational latency) + (transfer time)

$= T_s + {}^{1}\!/_{2R} + {}^{N_s}\!/_{(R \cdot N_t)}$

The first term is the mechanical seek, the second is the average wait for half a revolution before the target sector arrives, and the third is how long that sector takes to pass under the head at the disk's rotational speed.

CHECK YOUR UNDERSTANDING

1. Define seek time, rotational latency, and transfer time, and explain how each contributes to total disk access time. **[6M]**
2. Distinguish a hard disk from a floppy disk. **[2M]**
3. Why is magnetic tape a sequential-access device, and what trade-off does that bring compared to disks? **[4M]**
4. How do optical disks store information, and how does that differ physically from magnetic recording? **[4M]**
5. What is the transfer rate of an eight-track magnetic tape whose speed is 120 inches per second and density is 1600 bits per inch? **[4M]**

---

4. Associative Memory

Mano §12-4 (Hamacher has no dedicated section on this — it only uses associative searching inside cache, covered in Chapter 5)

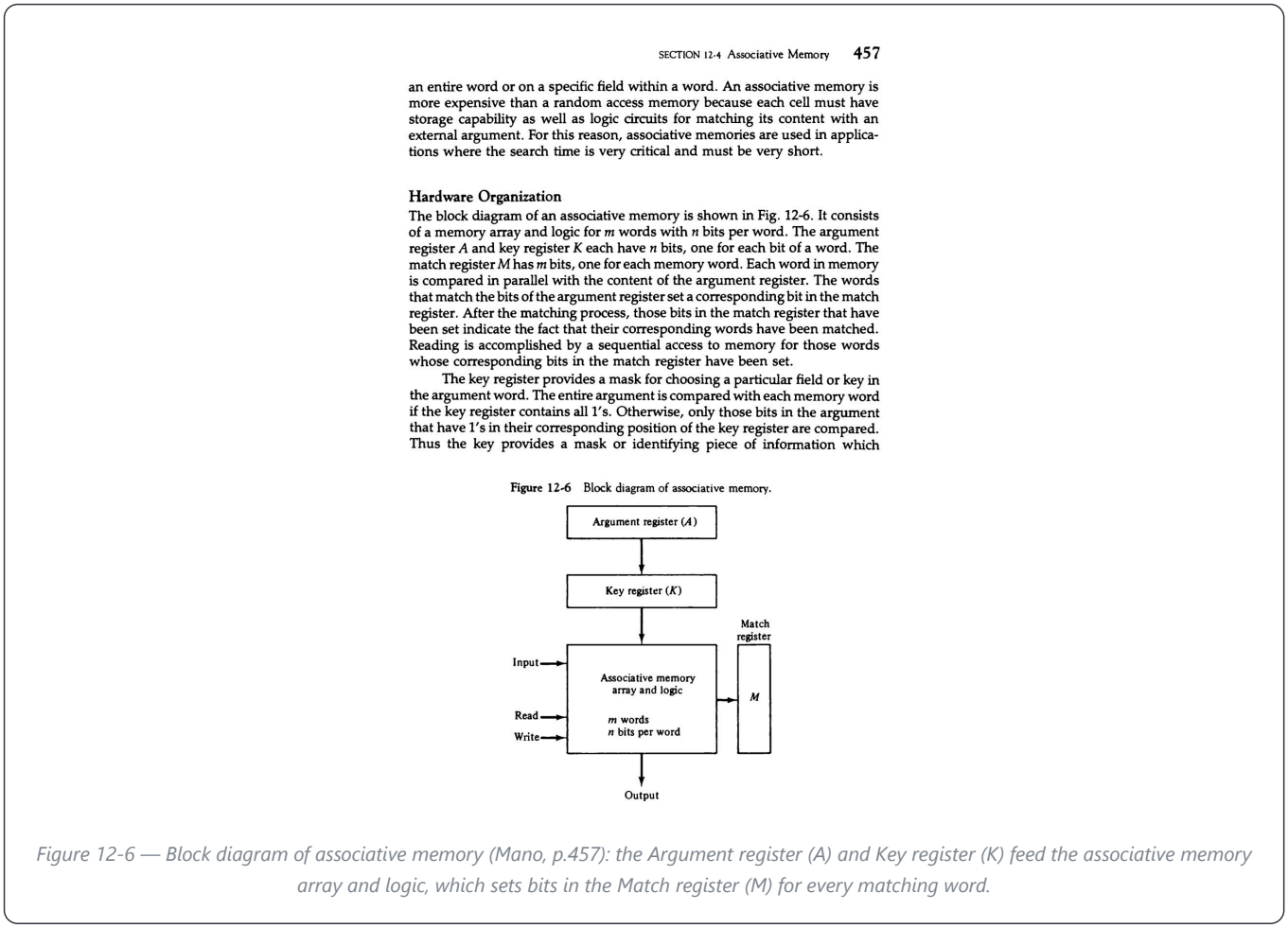
4.1 Why search by content instead of by address?

- Many data-processing applications require **searching a table** for an item, rather than reading a known address. The established way is to store the table sequentially and search it address by address — the number of accesses needed depends on the search algorithm and can be large.

- Search time can be reduced considerably if stored data can be identified for access by the **content** of the data itself, rather than by its address. A memory unit accessed this way is called an **associative memory** or **content-addressable memory (CAM)**.
- An associative memory is accessed **simultaneously and in parallel** on the basis of data content. When a word is written, **no address is given**; the memory finds an empty unused location by itself. To read, the **content of the word**, or part of it, is specified, and the memory locates all words that match and marks them for reading.
- Because every word is compared in parallel, associative memory is **more expensive** than ordinary RAM (each cell needs its own comparison logic as well as storage), so it is reserved for applications where search time is critical and must be very short.

4.2 Hardware organization

- An associative memory consists of a **memory array** (holding *m* words of *n* bits each), plus two special registers: the **argument register (A)**, holding the word being searched for, and the **key register (K)**, which masks which bits of the argument actually take part in the comparison.
- Every word in the array is compared, bit by bit and in parallel, against the argument register — but only in bit positions where the corresponding key-register bit is 1. If the key register is all 1s, the **entire** argument word is compared against every memory word.
- The **match register (M)** has one bit per memory word; after the comparison, M_i is set to 1 for every word i whose unmasked bits all match the argument — those words are then read out.



SECTION 12-4 Associative Memory 457

an entire word or on a specific field within a word. An associative memory is more expensive than a random access memory because each cell must have storage capability as well as logic circuits for matching its content with an external argument. For this reason, associative memories are used in applications where the search time is very critical and must be very short.

Hardware Organization

The block diagram of an associative memory is shown in Fig. 12-6. It consists of a memory array and logic for *m* words with *n* bits per word. The argument register *A* and key register *K* each have *n* bits, one for each bit of a word. The match register *M* has *m* bits, one for each memory word. Each word in memory is compared in parallel with the content of the argument register. The words that match the bits of the argument register set a corresponding bit in the match register. After the matching process, those bits in the match register that have been set indicate the fact that their corresponding words have been matched. Reading is accomplished by a sequential access to memory for those words whose corresponding bits in the match register have been set.

The key register provides a mask for choosing a particular field or key in the argument word. The entire argument is compared with each memory word if the key register contains all 1's. Otherwise, only those bits in the argument that have 1's in their corresponding position of the key register are compared. Thus the key provides a mask or identifying piece of information which

Figure 12-6 Block diagram of associative memory.

Figure 12-6 — Block diagram of associative memory (Mano, p.457): the Argument register (A) and Key register (K) feed the associative memory array and logic, which sets bits in the Match register (M) for every matching word.

4.3 Worked example — masked comparison

WORKED EXAMPLE (from Mano)

Suppose only the three left-most bits of A and K matter (K has 1s only in those positions):

```
A = 101 111100
K = 111 000000
Word 1 = 100 111100   →  no match  (first 3 bits differ: 101 vs 100)
Word 2 = 101 000001   →  match     (first 3 bits agree: 101 = 101; remaining bits are masked out by K and ignored)
```

Word 2 matches the unmasked argument field even though its last six bits look nothing like the argument's — because K = 0 in those positions, they simply aren't compared.

4.4 The cell array and match logic

- Each cell in the array is written C_{ij} — word *i*, bit *j*. A bit A_j of the argument is compared with every column *j* across all words, but only where $K_j = 1$.

specifies how the reference to memory is made. To illustrate with a numerical example, suppose that the argument register A and the key register K have the bit configuration shown below. Only the three leftmost bits of A are compared with memory words because K has 1's in these positions.

A	101 111100	
K	111 000000	
Word 1	100 111100	no match
Word 2	101 000001	match

Word 2 matches the unmasked argument field because the three leftmost bits of the argument and the word are equal.

The relation between the memory array and external registers in an associative memory is shown in Fig. 12-7. The cells in the array are marked by the letter C with two subcripts. The first subscript gives the word number and the second specifies the bit position in the word. Thus cell C_{ij} is the cell for bit j in word i. A bit A_j in the argument register is compared with all the bits in column j of the array provided that $K_j = 1$. This is done for all columns $j = 1, 2, \ldots, n$. If a match occurs between all the unmasked bits of the argument and the bits in word i, the corresponding bit M_i in the match register is set to 1. If one or more unmasked bits of the argument and the word do not match, M_i is cleared to 0.

Figure 12-7 Associative memory of m word, n cells per word.

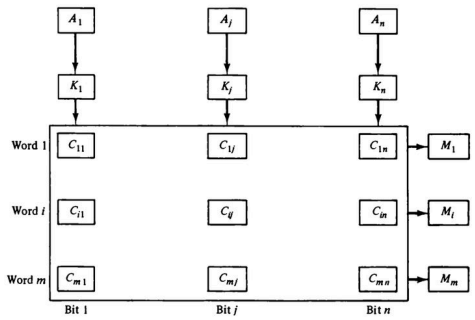


Figure 12-7 — Associative memory of m words, n cells per word (Mano, p.458). Each column j receives argument bit A_j and key bit K_j; each cell C_ij feeds its word's match bit M_i.

- The match logic for one word compares, bit by bit, the stored bit F_{ij} against the argument bit A_j, but the key bit K_j can force a position to be treated as an automatic match: the comparison term is $x_j + K_j'$, so when $K_j = 0$ the term is forced to 1 (always "equal") regardless of the actual bit values.
- The full match equation for word *i* is the AND of every bit position's term: $M_i = \Pi_{j=1..n} (A_jF_{ij} + A_j'F_{ij}' + K_j')$ — every unmasked bit must agree for M_i to become 1.

The internal organization of a typical cell C_{ij} is shown in Fig. 12-8. It consists of a flip-flop storage element F_{ij} and the circuits for reading, writing, and matching the cell. The input bit is transferred into the storage cell during a write operation. The bit stored is read out during a read operation. The match logic compares the content of the storage cell with the corresponding unmasked bit of the argument and provides an output for the decision logic that sets the bit in M_i.

Match Logic

The match logic for each word can be derived from the comparison algorithm for two binary numbers. First, we *neglect* the key bits and compare the argument in A with the bits stored in the cells of the words. Word i is equal to the argument in A if $A_j = F_{ij}$ for $j = 1, 2, \ldots, n$. Two bits are equal if they are both 1 or both 0. The equality of two bits can be expressed logically by the Boolean function

$$x_j = A_j F_{ij} + A_j' F_{ij}'$$

where $x_j = 1$ if the pair of bits in position j are equal; otherwise, $x_j = 0$.

For a word i to be equal to the argument in A we must have all x_j variables equal to 1. This is the condition for setting the corresponding match bit M_i to 1. The Boolean function for this condition is

$$M_i = x_1 x_2 x_3 \cdots x_n$$

and constitutes the AND operation of all pairs of matched bits in a word.

Figure 12-8 One cell of associative memory.

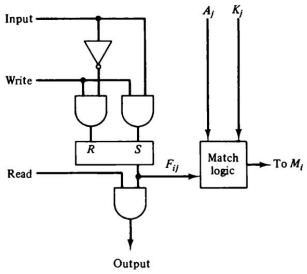


Figure 12-8 — One cell of associative memory (Mano, p.459). A flip-flop F_ij stores the bit; read/write/match logic compares the stored value against A_j and K_j to help set M_i.

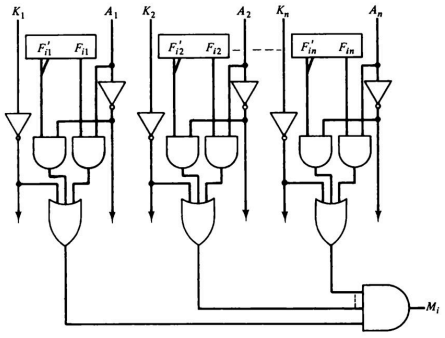


Figure 12-9 Match logic for one word of associative memory.

In most applications, the associative memory stores a table with no two identical items under a given key. In this case, only one word may match the unmasked argument field. By connecting output M_i directly to the read line in the same word position (instead of the M register), the content of the matched word will be presented automatically at the output lines and no special read command signal is needed. Furthermore, if we exclude words having a zero content, an all-zero output will indicate that no match occurred and that the searched item is not available in memory.

Write Operation

An associative memory must have a write capability for storing the information to be searched. Writing in an associative memory can take different forms, depending on the application. If the entire memory is loaded with new information at once prior to a search operation then the writing can be done by addressing each location in sequence. This will make the device a random-access memory for writing and a content addressable memory for reading. The advantage here is that the address for input can be decoded as in a random-access memory. Thus instead of having m address lines, one for each word in memory, the number of address lines can be reduced by the decoder to d lines, where $m = 2^d$.

Figure 12-9 — Match logic for one word of associative memory (Mano, p.461). Each column contributes an AND/OR/inverter term; all n terms feed a common AND gate producing that word's match bit M_i.

4.5 Reading, writing, and tag bits

- **Read:** if more than one word matches, the matched words are read out one at a time by scanning the match register. If the memory excludes an all-zero word from being a valid entry, an all-zero output on the read lines itself signals "not found."
- **Write:** entries can be loaded in sequence like ordinary random-access memory (decoding d address lines to select 1-of-2^d words), making the device a normal RAM for *writing* and a CAM only for *reading*.
- Where words are added and deleted at different times, a **tag register** (one bit per word) distinguishes active from inactive entries — a word is deleted simply by clearing its tag bit, and a new word is stored in the first location whose tag bit reads 0.

CHECK YOUR UNDERSTANDING

1. Define associative memory / content-addressable memory, and explain how it differs from ordinary random-access memory. **[4M]**
2. What are the argument register and key register used for, and what happens when the key register is all zeros? **[6M]**
3. Draw the block diagram of associative memory and label the match register. **[6M]**
4. Obtain the match logic function M_i for one word of associative memory, including the effect of the key bits. **[6M]**
5. What additional logic is needed to give a no-match result for a word in associative memory when all its key bits are zero? **[4M]**
6. Explain how a tag register supports writing into an associative memory whose entries are added and deleted at different times. **[4M]**

---

5. Cache Memory

Hamacher §8.6

5.1 Why cache works: locality of reference

- Analysis of typical programs shows that references to memory at any given interval of time tend to be confined to a few localized areas — this is **locality of reference**. Loops and subroutines cause instruction references to cluster; iterative table-lookup procedures and arrays cause data references to cluster.
- If the actively-used portions of a program and its data are placed in a small, fast memory, average memory access time can be reduced considerably — this fast memory is the **cache**, placed between the CPU and main memory. Cache access time is typically **5–10×** faster than main memory.
- **Operation:** when the CPU needs a word, the cache is checked first. If found — a **hit** — it is read directly from the cache. If not found — a **miss** — a whole **block** of words containing the needed one is brought in from main memory to the cache, and the CPU reads it.

Hit ratio: the fraction of all memory references found in the cache. Ratios of 0.9 and higher are commonly reported for real programs, which is why even a small cache produces a large drop in **average memory access time**.

5.2 Mapping functions

- The transformation of data from main memory to cache is called a **mapping** process. Three mapping techniques matter:
- **1. Direct mapping** — each main-memory block can go in exactly **one, specific** cache line, determined by (block address) mod (number of cache lines). Simple and fast, but two frequently-used blocks that map to the *same* line will repeatedly evict each other (thrashing).

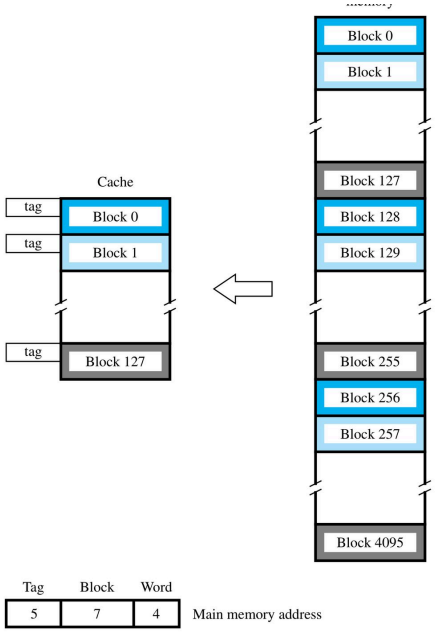


Figure 8.16 Direct-mapped cache.

ese blocks must be transferred to the block-1 position in the cache. Co
›y allowing the new block to overwrite the currently resident block.
direct mapping, the replacement algorithm is trivial. Placement of a b
etermined by its memory address. The memory address can be divide
shown in Figure 8.16. The low-order 4 bits select one of 16 words
:w block enters the cache, the 7-bit cache block field determines the cac

Figure 8.16 — Direct-mapped cache (Hamacher, p.291): the low-order bits of a block address pick one specific cache line, and a tag field identifies which of the several candidate blocks currently resides there.

- **2. Associative (fully associative) mapping** — a main-memory block may be placed in **any** cache line. Most flexible, fewest conflicts, but every line's tag must be searched (an associative search — this is exactly the associative memory of Chapter 4, applied to cache tags) to find a match, which is more complex/costly hardware.

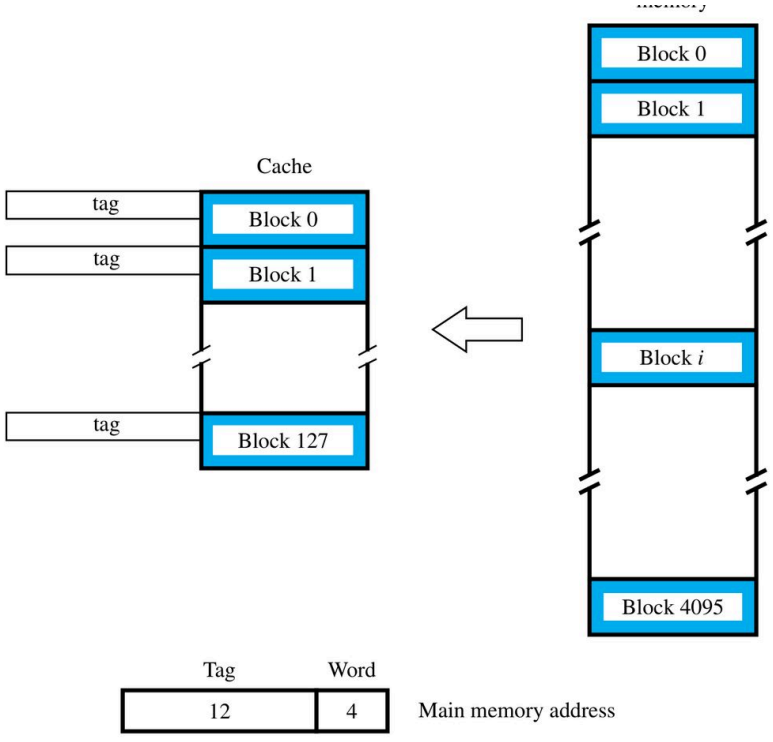


Figure 8.17 Associative-mapped cache.

Figure 8.17 — Associative-mapped cache (Hamacher, p.293): the full tag of every cache line is compared in parallel against the address; a match locates the block wherever it happens to sit.

- **3. Set-associative mapping** — a compromise: the cache is divided into **sets**; a block maps to exactly one **set** (like direct mapping) but can occupy **any line within that set** (like associative mapping, but only searching a small set instead of the whole cache). A cache with *k* blocks per set is called **k-way set-associative**.

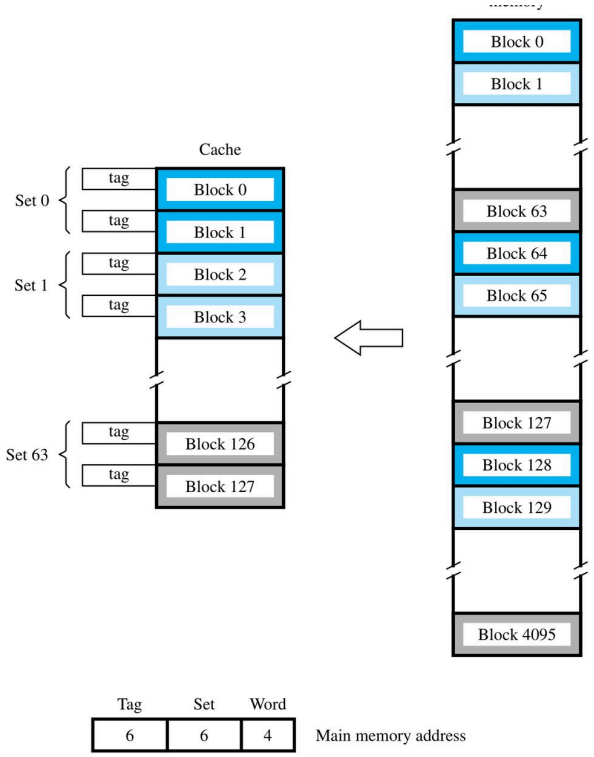


Figure 8.18 Set-associative-mapped cache with two blocks per set.

Figure 8.18 — Set-associative-mapped cache with two blocks per set (Hamacher, p.294): the index selects a set of two lines, and the tags of just those two lines are compared against the address.

5.3 Replacement and write policies

- **Replacement policy** (needed whenever a miss occurs and the target set/line is full): **Random** (pick any line), **FIFO** (evict the block that has been in the cache the longest), or **LRU — Least Recently Used** (evict the block that has gone the longest without being referenced; usually implemented with small "aging" counters per line). Direct mapping needs no replacement policy at all — there is only ever one candidate line.
- **Write-through:** every write updates both cache and main memory immediately. Simple, keeps memory always consistent, but generates more memory traffic.
- **Write-back:** a write updates only the cache; the modified block is written to main memory only when it is replaced (flagged with a per-line **dirty/valid bit**). Less memory traffic, but a more complex control scheme, and any device sharing memory via DMA might see stale data.
- A **valid bit** per cache line handles initialization: all valid bits are cleared to 0 at power-on so the cache starts empty rather than serving garbage data as if it were a hit.

WORKED EXAMPLE

Q. A cache holds 512 lines of 16 words each and is backed by a 4M-word main memory (24-bit address). Give the tag/index/word-offset field widths for (a) direct mapping and (b) 4-way set-associative mapping.

A. Word offset (within a 16-word block) needs 4 bits.
(a) Direct: index selects 1 of 512 lines ⇒ 9 bits; remaining 24−9−4 = **11 tag bits**.
(b) 4-way set-associative: 512 lines / 4 per set = 128 sets ⇒ index is 7 bits; remaining 24−7−4 = **13 tag bits**, and all 4 tags in the selected set are compared in parallel against those 13 bits.

CHECK YOUR UNDERSTANDING

1. Explain direct-mapped, fully-associative, and set-associative cache organization, with one advantage of each. **[6M]**
2. Compare write-through and write-back cache policies in terms of memory traffic and consistency. **[4M]**
3. Define hit ratio and explain why a high hit ratio matters even though the cache is much smaller than main memory. **[4M]**
4. Why does direct mapping need no replacement algorithm, while associative and set-associative mapping do? **[4M]**
5. A block-set-associative cache consists of 64 blocks divided into 4-block sets. Main memory contains 4096 blocks, 128 words per block. How many bits are in the tag, set, and word fields? **[6M]**

---

6. Virtual Memory

Hamacher §8.8 · Mano §12-6

6.1 The big idea

- **Virtual memory** gives every program the illusion of a large, private address space, even though physical main memory is much smaller. Programs, and the data they use, are first held in **auxiliary memory**; portions are brought into main memory only as they are actually needed by the CPU.
- An address used by a program is a **virtual address**, drawn from the **address space**. An address in main memory is called a **physical address** (or location), drawn from the **memory space**. In most modern computers the address space is allowed to be **larger** than the memory space.
- Every address the CPU generates goes through a **mapping** (translation) process, done **dynamically** in hardware, that converts the virtual address to a physical address **as the reference is made**.

6.2 Pages, blocks, and page faults

- To make the mapping practical, both address space and memory space are divided into groups of equal size: fixed-length groups in address space are called **pages**; the corresponding fixed-length groups in memory space are called **blocks** (or **page frames**). Page/block size commonly ranges from 64 to 4096 words.
- A **page fault** occurs when the CPU references a page that is **not currently in main memory**. The hardware/OS then suspends the running program, brings the required page in from auxiliary memory (basically an I/O operation), and resumes — possibly first evicting another page to make room.
- Choosing a **page size** is a trade-off: very small pages waste table space and increase the fraction of accesses that miss; very large pages mean a program may not use all of a page it has loaded, wasting main-memory space.

6.3 The page table (address translation)

- A **page table** maps each virtual page number to its corresponding physical **page frame**. A virtual address is split into a **page number** field and an **offset (line number)** field; the page number indexes the page table to find the frame, and the frame is combined with the unchanged offset to form the physical address.
- Each page-table entry also carries a **presence/valid bit** indicating whether that page is currently resident in main memory at all — if not, referencing it triggers a page fault.

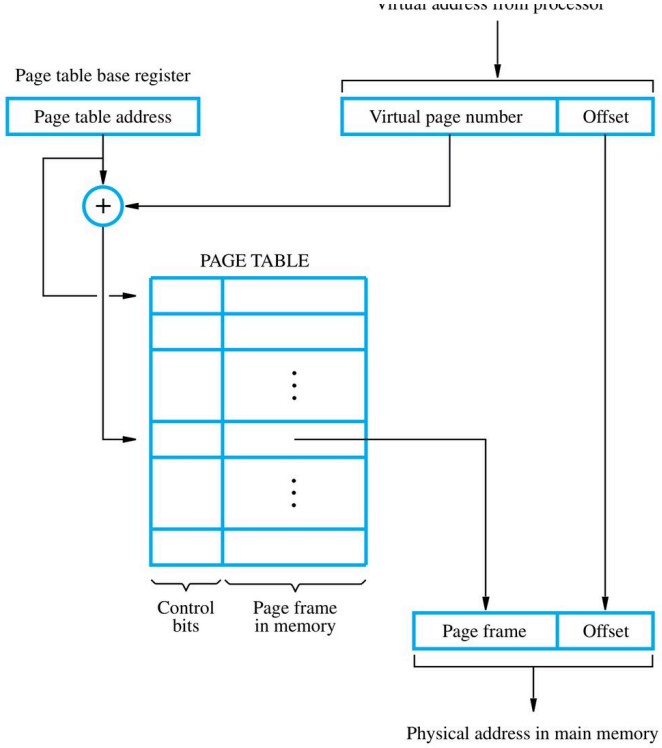


Figure 8.25 Virtual-memory address translation.

Figure 8.25 — Virtual-memory address translation (Hamacher, p.307): the virtual page number indexes the page table (added to a page-table base address); the resulting page frame, combined with the offset, gives the physical address.

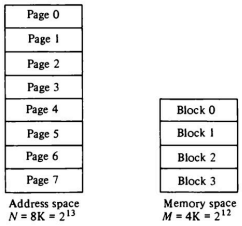


Figure 12-18 Address space and memory space split into groups of 1K words.

word in the memory page table at the page number address is read out into the memory table buffer register. If the presence bit is a 1, the block number thus read is transferred to the two high-order bits of the main memory address register. The line number from the virtual address is transferred into the 10 low-order bits of the memory address register. A read signal to main memory

Figure 12-19 Memory table in a paged system.

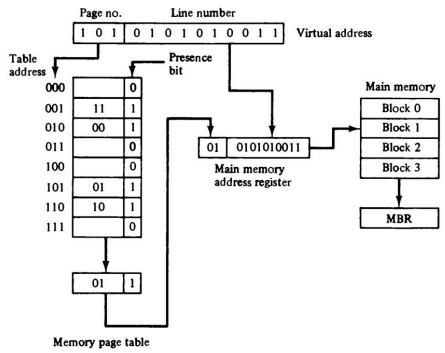


Figure 12-18/12-19 — Splitting address space and memory space into equal-size pages/blocks, and a memory-page table in a paged system (Mano, p.473): each page-table entry gives a block number and a presence bit.

6.4 Associative memory as a fast page table

- A conventional random-access page table is inefficient: it needs one entry for **every** page even though, at any time, only a handful of pages are actually resident (blocks are far fewer than pages). Most of the table sits empty.
- A more efficient scheme stores the page table itself as an **associative memory**, with only as many entries as there are blocks in main memory: each word holds a **page number together with its block number**. The virtual page number is presented as the search argument; a match returns the block number directly — no match means a page fault.

transfers the content of the word to the main memory buffer register ready to be used by the CPU. If the presence bit in the word read from the page table is 0, it signifies that the content of the word referenced by the virtual address does not reside in main memory. A call to the operating system is then generated to fetch the required page from auxiliary memory and place it into main memory before resuming computation.

Associative Memory Page Table

A random-access memory page table is inefficient with respect to storage utilization. In the example of Fig. 12-19 we observe that eight words of memory are needed, one for each page, but at least four words will always be marked empty because main memory cannot accommodate more than four blocks. In general, a system with n pages and m blocks would require a memory-page table of n locations of which up to m blocks will be marked with block numbers and all others will be empty. As a second numerical example, consider an address space of 1024K words and memory space of 32K words. If each page or block contains 1K words, the number of pages is 1024 and the number of blocks 32. The capacity of the memory-page table must be 1024 words and only 32 locations may have a presence bit equal to 1. At any given time, at least 992 locations will be empty and not in use.

A more efficient way to organize the page table would be to construct it with a number of words equal to the number of blocks in main memory. In this way the size of the memory is reduced and each location is fully utilized. This method can be implemented by means of an associative memory with each word in memory containing a page number together with its corresponding

Figure 12-20 An associative memory page table.

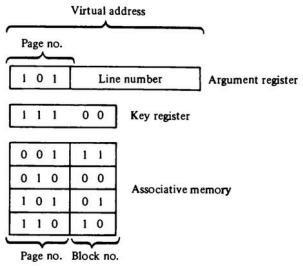


Figure 12-20 — An associative memory page table (Mano, p.474): the argument register holds the page number; each associative-memory word stores a (page number, block number) pair, sized to the number of blocks rather than the number of pages.

6.5 Page replacement

- When a page fault occurs and main memory is full, a page must be evicted to make room, chosen by a **replacement policy** — the same two classic choices as cache: **FIFO** (evict the page that has been resident longest) and **LRU** (evict the page least recently referenced, tracked with per-page aging counters).
- Virtual memory and cache memory rest on the **same underlying principle** — locality of reference — applied at different levels of the hierarchy: cache bridges the speed gap between processor and main memory (implemented in hardware), while virtual memory bridges the size/speed gap between main memory and secondary storage (implemented mostly in software, with hardware assistance).

WORKED EXAMPLE (Mano's numerical example, adapted)

Q. A computer has a main-memory capacity of 32K words (K = 1024) and an address space of 1024K words. Give N (address space size), M (memory space size), and the number of bits needed for a virtual address and a physical address.

A. N = 1024K = 2^{20}, so a virtual address needs **20 bits**. M = 32K = 2^{15}, so a physical address needs **15 bits**. Since the CPU only ever generates 20-bit virtual addresses, the memory-mapping hardware must translate every one of them into a 15-bit physical address before main memory is actually accessed.

CHECK YOUR UNDERSTANDING

1. Distinguish address space from memory space, and virtual address from physical address. **[4M]**
2. What is a page fault, and what does the operating system do when one occurs? **[4M]**
3. Describe, step by step, how the MMU/page table translates a virtual address into a physical address. **[6M]**
4. Why is a random-access page table inefficient compared to an associative-memory page table, for typical page/block counts? **[6M]**
5. Compare FIFO and LRU page-replacement policies. **[4M]**
6. Explain in what sense cache memory and virtual memory are "conceptually similar" techniques. **[4M]**

---

7. Memory Management Hardware

Mano §12-7 · Hamacher's brief MMU/TLB references

7.1 What a memory management unit does

- In a multiprogramming environment, many programs reside in memory simultaneously, and the amount of memory in use by any one program needs to change as it runs. A **memory management unit (MMU)** is the collection of hardware and software procedures that manages this.
- Its basic components: **(1)** a facility for **dynamic storage relocation** that maps logical (program-generated) addresses into physical memory addresses; **(2)** provision for **sharing** common programs among different users without keeping multiple copies in memory; **(3) protection** of information against unauthorized access, and prevention of one user's program from changing another's.
- The address generated by a segmented program is called a **logical address**. It is similar to a virtual address, except that logical address space is organized into variable-length **segments** rather than uniform, fixed-length pages.

7.2 Segments, and segmented-page mapping

- A **segment** is a set of logically related instructions or data given a name — e.g. a subroutine, an array, a table, or a whole user program. Segments let several users share a single copy of a common program (such as a compiler) while each keeps a private copy of their own data.
- In **segmented-page mapping**, a logical address is split into three fields: **segment**, **page**, and **word**. The segment number indexes a **segment table**, which points to the base of that segment's own **page table**; the page number then indexes into that page table to obtain a block number, which combines with the word offset to give the physical address.
- Because this scheme needs **two** extra memory accesses (segment table, then page table) before the real memory access, a fast **associative memory** is used to cache the most recently used (segment, page) → block translations. This is called a **Translation Look-aside Buffer (TLB)**. On a TLB hit the mapping is available immediately; only on a TLB miss is the slower two-table walk performed.

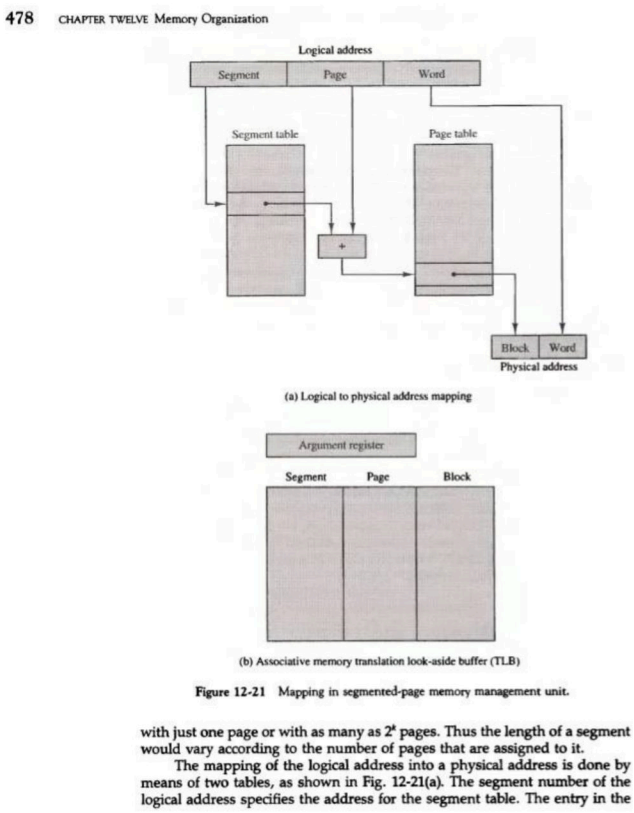


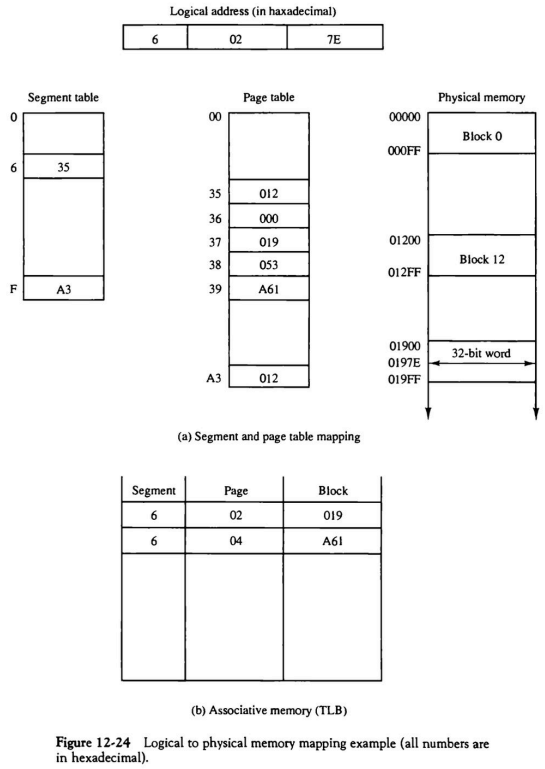
Figure 12-21 — Mapping in a segmented-page memory management unit (Mano, p.478): (a) logical-to-physical address mapping via a segment table and page table; (b) the associative-memory TLB, indexed by segment and page, that shortcuts this walk.

7.3 Worked numerical example (Mano's example, adapted)

WORKED EXAMPLE

A logical address specifies 16 possible segments, each of up to 256 pages of 256 words each (logical address = 4-bit segment + 8-bit page + 8-bit word = 20 bits). Physical memory is 2^{20} words, divided into 4096 blocks of 256 words each (physical address = 12-bit block + 8-bit word).

Suppose logical address **6 / 02 / 7E** (segment 6, page 2, offset 7E hex) is referenced. The segment table says segment 6's page table starts at address 35; that page table's entry for page 2 (at address 35+2=37) gives block number 019. So the physical address becomes **block 019, offset 7E** — and because segment 6/page 2 was just referenced, the pair (6,2)→019 is also written into the associative TLB, so the *next* reference to segment 6 page 2 skips the table walk entirely.



(a) Segment and page table mapping

Segment	Page	Block
6	02	019
6	04	A61

(b) Associative memory (TLB)

Figure 12-24 Logical to physical memory mapping example (all numbers are in hexadecimal).

Figure 12-24 — Logical-to-physical memory mapping example, all numbers in hexadecimal (Mano, p.481): the segment table and page table together resolve logical address 6/02/7E to physical block 019; the TLB caches this and one other recently-used (segment,page) pair.

7.4 Memory protection

- Because memory is shared and moved around dynamically, **protection** must accompany every access. Rather than protecting the physical address directly (awkward, since blocks move), protection bits are attached to each **segment-table** or **page-table** entry — i.e. in the logical-address space, which stays fixed for a program even as its physical location changes.
- A **segment descriptor** typically carries: a **base address** field (pointing to the segment's page-table base, or block base), a **length** field (checked against the page/offset to catch out-of-range accesses — a size violation), and a **protection** field.

12-24(b) shows that pages 2 and 4 of segment 6 have been referenced previously and therefore their corresponding block numbers are stored in the associative memory.

From this example it should be evident that the memory management system can assign any number of pages to each segment. Each logical page can be mapped into any block in physical memory. Pages can move to different blocks in memory depending on memory space requirements. The only updating required is the change of the block number in the page table. Segments can grow or shrink independently without affecting each other. Different segments can use the same block of memory if it is required to share a program by many users. For example, block number 12 in physical memory can be assigned a second logical address F0000 through F00FF. This specifies segment number 15 and page 0, which maps to block 12 as shown in Fig. 12-24(a).

Memory Protection

Memory protection can be assigned to the physical address or the logical address. The protection of memory through the physical address can be done by assigning to each block in memory a number of protection bits that indicate the type of access allowed to its corresponding block. Every time a page is moved from one block to another it would be necessary to update the block protection bits. A much better place to apply protection is in the logical address space rather than the physical address space. This can be done by including protection information within the segment table or segment register of the memory management hardware.

The content of each entry in the segment table or a segment register is called a descriptor. A typical descriptor would contain, in addition to a base address field, one or two additional fields for protection purposes. A typical format for a segment descriptor is shown in Fig. 12-25. The base address field gives the base of the page table address in a segmented-page organization or the block base address in a segment register organization. This is the address used in mapping from a logical to the physical address. The length field gives the segment size by specifying the maximum number of pages assigned to the segment. The length field is compared against the page number in the logical address. A size violation occurs if the page number falls outside the segment length boundary. Thus a given program and its data cannot access memory not assigned to it by the operating system.

The protection field in a segment descriptor specifies the access rights available to the particular segment. In a segmented-page organization, each

Figure 12-25 Format of a typical segment descriptor.



Figure 12-25 — Format of a typical segment descriptor (Mano, p.482): base address, length, and protection fields together let the MMU both translate and validate every access.

- Common protection levels an operating system assigns via the descriptor: **full read/write** (given to a program for its own instructions/data), **read-only** (for shared utility/library routines so many programs can read them but none can modify them), **execute-only** (a segment can be fetched as instructions but not read as data, protecting compiled code from being copied), and **system-only** (reserved for operating-system segments, inaccessible to ordinary user programs).

CHECK YOUR UNDERSTANDING

1. List the three basic components/responsibilities of a memory management unit. **[3M]**
2. What is a Translation Look-aside Buffer, and why is associative memory well suited to implementing it? **[6M]**
3. Distinguish a logical address (in a segmented system) from a virtual address (in a pure paging system). **[4M]**
4. Trace how a segmented-page MMU translates a logical address into a physical address when the TLB misses. **[6M]**
5. What fields make up a segment descriptor, and what does each protect against? **[4M]**
6. Why is protection information attached to the logical address (segment/page table) rather than to the physical address directly? **[4M]**

Sources: Hamacher, Vranesic & Zaky, *Computer Organization and Embedded Systems*, 5th ed., McGraw-Hill, Chapter 8 "The Memory System" · M. Morris Mano, *Computer System Architecture*, PHI, Chapter 12 "Memory Organization." Figures are cropped directly from these two texts. Direct Memory Access (Hamacher Figs 8.12/8.13) is intentionally excluded — it belongs to Unit 5 (Input and Output Organizations) of this syllabus.